2048game Project Documentation

1. Project Content

An enhanced version of the classic 2048 game implemented in Python using Pygame. Features multiple levels with distinct target tiles, merge sound effects, and a settings menu to adjust controls (Arrows/WASD) and volume for replayability.

2. Project Code

```
import pygame

import random


# Initialize Pygame & Mixer

pygame.init()

pygame.mixer.init()


# Constants

SIZE         = WIDTH, HEIGHT = 400, 550

GRID_SIZE    = 4

TILE_SIZE    = WIDTH // GRID_SIZE

MARGIN       = 5

FONT         = pygame.font.SysFont("arial", 36)

SMALL_FONT   = pygame.font.SysFont("arial", 24)


# Colors

BG_COLOR     = (187, 173, 160)

EMPTY_COLOR  = (205, 193, 180)

BUTTON_COLOR = (100, 100, 100)
```

```
BUTTON_HOVER  = (150, 150, 150)
TEXT_COLOR    = (0, 0, 0)

TILE_COLORS = {
   2:    (238, 228, 218),
   4:    (237, 224, 200),
   8:    (242, 177, 121),
   16:   (245, 149, 99),
   32:   (246, 124, 95),
   64:   (246, 94, 59),
   128:  (237, 207, 114),
   256:  (237, 204, 97),
   512:  (237, 200, 80),
   1024: (237, 197, 63),
   2048: (237, 194, 46),
}

# Load merge sound
merge_sound = pygame.mixer.Sound("merge.wav")
merge_sound.set_volume(0.5)

# Screen setup
screen = pygame.display.set_mode(SIZE)
pygame.display.set_caption("2048 with Levels")

# Control schemes
```

```
CONTROL_SCHEMES = {
   "Arrows": {"up":pygame.K_UP,    "down":pygame.K_DOWN,
            "left":pygame.K_LEFT,"right":pygame.K_RIGHT},
   "WASD":   {"up":pygame.K_w,     "down":pygame.K_s,
            "left":pygame.K_a,   "right":pygame.K_d},
}
controls = CONTROL_SCHEMES["Arrows"]
volume   = 0.5

# Levels: list of target tiles per level
LEVELS = [
   [128],        # Level 1 target
   [256, 512],    # Level 2 targets
   [1024, 2048],  # Level 3 targets
]

def draw_button(rect, text):
   mouse = pygame.mouse.get_pos()
   click = pygame.mouse.get_pressed()[0]
   color = BUTTON_HOVER if rect.collidepoint(mouse) else BUTTON_COLOR
   pygame.draw.rect(screen, color, rect)
   label = SMALL_FONT.render(text, True, TEXT_COLOR)
   screen.blit(label, label.get_rect(center=rect.center))
   return rect.collidepoint(mouse) and click

def init_grid():
```

```
    grid = [[0]*GRID_SIZE for _ in range(GRID_SIZE)]
    add_new_tile(grid); add_new_tile(grid)
    return grid

def add_new_tile(grid):
    empties = [(i,j) for i in range(GRID_SIZE) for j in range(GRID_SIZE) if grid[i][j]==0]
    if empties:
        i,j = random.choice(empties)
        grid[i][j] = random.choice([2,4])

def draw_grid(grid, score, level_idx):
    screen.fill(BG_COLOR)
    # Header
    screen.blit(SMALL_FONT.render(f"Score: {score}", True, TEXT_COLOR), (10,10))
    screen.blit(SMALL_FONT.render(f"Level: {level_idx+1}", True, TEXT_COLOR), (WIDTH-
110,10))
    # Tiles
    for i in range(GRID_SIZE):
        for j in range(GRID_SIZE):
            val = grid[i][j]
            rect = pygame.Rect(
                j*TILE_SIZE + MARGIN,
                i*TILE_SIZE + MARGIN + 50,
                TILE_SIZE - 2*MARGIN,
                TILE_SIZE - 2*MARGIN
            )
```

```
            pygame.draw.rect(screen, TILE_COLORS.get(val, EMPTY_COLOR), rect)
            if val:
                lbl = FONT.render(str(val), True, TEXT_COLOR)
                screen.blit(lbl, lbl.get_rect(center=rect.center))
    pygame.display.flip()

def compress(grid):
    new = [[0]*GRID_SIZE for _ in range(GRID_SIZE)]
    for i in range(GRID_SIZE):
        pos = 0
        for j in range(GRID_SIZE):
            if grid[i][j]:
                new[i][pos] = grid[i][j]; pos += 1
    return new

def merge(grid):
    gain = 0
    for i in range(GRID_SIZE):
        for j in range(GRID_SIZE-1):
            if grid[i][j] and grid[i][j]==grid[i][j+1]:
                grid[i][j] *= 2
                grid[i][j+1] = 0
                gain += grid[i][j]
                merge_sound.play()
    return grid, gain
```

```
def reverse(grid):
    return [list(reversed(r)) for r in grid]

def transpose(grid):
    return [list(r) for r in zip(*grid)]

def move_left(grid):
    c = compress(grid)
    m, g = merge(c)
    return compress(m), g

def move_right(grid):
    r = reverse(grid)
    m, g = move_left(r)
    return reverse(m), g

def move_up(grid):
    t = transpose(grid)
    m, g = move_left(t)
    return transpose(m), g

def move_down(grid):
    t = transpose(grid)
    m, g = move_right(t)
    return transpose(m), g
```

```
def is_over(grid):
    for i in range(GRID_SIZE):
        for j in range(GRID_SIZE):
            if grid[i][j]==0: return False
            if j<GRID_SIZE-1 and grid[i][j]==grid[i][j+1]: return False
            if i<GRID_SIZE-1 and grid[i][j]==grid[i+1][j]: return False
    return True


def show_screen(message, final_score, button_text="Play Again"):
    btn = pygame.Rect(WIDTH//2-60, HEIGHT-100, 120, 40)
    while True:
        screen.fill(BG_COLOR)
        screen.blit(FONT.render(message, True, TEXT_COLOR),
                    FONT.render(message, True, TEXT_COLOR).get_rect(center=(WIDTH//2,
HEIGHT//2-40)))
        screen.blit(SMALL_FONT.render(f"Score: {final_score}", True, TEXT_COLOR),
                    SMALL_FONT.render(f"Score: {final_score}", True,
TEXT_COLOR).get_rect(center=(WIDTH//2, HEIGHT//2)))
        if draw_button(btn, button_text):
            pygame.time.wait(200)
            return True
        for e in pygame.event.get():
            if e.type==pygame.QUIT:
                pygame.quit(); exit()
        pygame.display.flip()
        pygame.time.Clock().tick(30)
```

```
def settings_menu():
    global controls, volume
    btn_back     = pygame.Rect(10, HEIGHT-50,     80, 40)
    btn_toggle    = pygame.Rect(WIDTH//2-60, HEIGHT//2-10, 120, 40)
    btn_vol_minus = pygame.Rect(50, HEIGHT//2+50,  40, 40)
    btn_vol_plus  = pygame.Rect(WIDTH-90, HEIGHT//2+50, 40, 40)


    while True:
        screen.fill(BG_COLOR)
        screen.blit(FONT.render("Settings", True, TEXT_COLOR), (WIDTH//2-60, 20))
        # Controls
        scheme = "WASD" if controls==CONTROL_SCHEMES["WASD"] else "Arrows"
        screen.blit(SMALL_FONT.render(f"Controls: {scheme}", True, TEXT_COLOR), (50,100))
        if draw_button(btn_toggle, "Toggle Keys"):
            controls = (CONTROL_SCHEMES["WASD"] if
controls==CONTROL_SCHEMES["Arrows"]
                        else CONTROL_SCHEMES["Arrows"])
            pygame.time.wait(200)
        # Volume
        screen.blit(SMALL_FONT.render(f"Volume: {int(volume*100)}%", True, TEXT_COLOR),
(WIDTH//2-50, 150))
        if draw_button(btn_vol_minus, "-"):
            volume = max(0, volume-0.1)
            merge_sound.set_volume(volume)
            pygame.time.wait(150)
        if draw_button(btn_vol_plus, "+"):
            volume = min(1, volume+0.1)
```

```
            merge_sound.set_volume(volume)
            pygame.time.wait(150)
        if draw_button(btn_back, "Back"):
            pygame.time.wait(200)
            return
        for e in pygame.event.get():
            if e.type==pygame.QUIT:
                pygame.quit(); exit()
        pygame.display.flip()
        pygame.time.Clock().tick(30)

def main_menu():
    btn_play     = pygame.Rect(WIDTH//2-60, HEIGHT//2-20, 120, 40)
    btn_settings = pygame.Rect(WIDTH//2-60, HEIGHT//2+40, 120, 40)
    while True:
        screen.fill(BG_COLOR)
        screen.blit(FONT.render("2048 Game", True, TEXT_COLOR), (WIDTH//2-80, 100))
        if draw_button(btn_play, "Play"):
            pygame.time.wait(200)
            return
        if draw_button(btn_settings, "Settings"):
            pygame.time.wait(200)
            settings_menu()
        for e in pygame.event.get():
            if e.type==pygame.QUIT:
                pygame.quit(); exit()
```

```
        pygame.display.flip()
        pygame.time.Clock().tick(30)

def play_game():
    for idx, targets in enumerate(LEVELS):
        grid = init_grid()
        score = 0
        draw_grid(grid, score, idx)
        while True:
            for e in pygame.event.get():
                if e.type==pygame.QUIT:
                    pygame.quit(); exit()
                if e.type==pygame.KEYDOWN:
                    if e.key==controls["left"]:
                        new, gain = move_left(grid)
                    elif e.key==controls["right"]:
                        new, gain = move_right(grid)
                    elif e.key==controls["up"]:
                        new, gain = move_up(grid)
                    elif e.key==controls["down"]:
                        new, gain = move_down(grid)
                    else:
                        continue
                    if new!=grid:
                        grid = new; score+=gain
                        add_new_tile(grid)
```

```
                draw_grid(grid, score, idx)
        # Level complete?
        if any(cell in targets for row in grid for cell in row):
            if idx < len(LEVELS)-1:
                show_screen(f"Level {idx+1} Complete", score, button_text="Next Level")
                break
            else:
                replay = show_screen("You Win!", score)
                return replay
        # Game over?
        if is_over(grid):
            replay = show_screen("Game Over", score)
            return replay
        pygame.time.wait(100)
    return True

if __name__ == "__main__":
    while True:
        main_menu()
        replay = play_game()
        if not replay:
            break
    pygame.quit()
```

3. Key Technologies

• Python 3

• Pygame Library

• pygame.mixer for sound effects

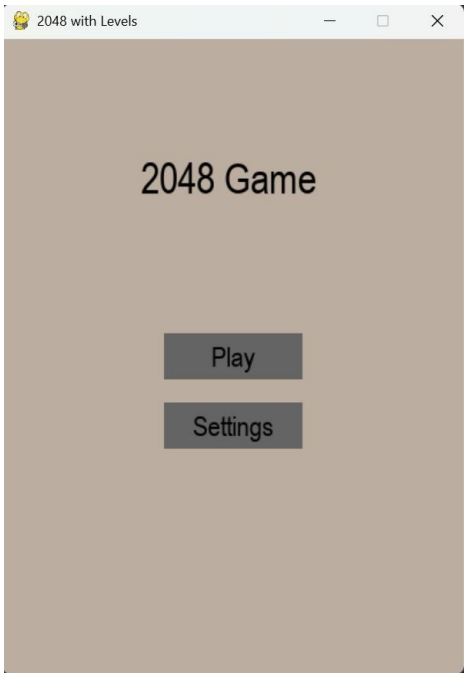
• Level management and state handling

4. Description

Players slide numbered tiles on a 4×4 grid using keyboard controls to combine matching tiles. Each level has specific target tiles to reach. Sound effects play on each merge. Settings menu allows control scheme (Arrows/WASD) and volume adjustment.

5. Output

Test case-1



Test case-2

6. Further Research

• Add undo move feature and hint system

• Implement AI solver for optimal moves

• Introduce power-ups or special tile types

• Develop mobile version using Kivy

• Integrate online leaderboards

3D Button Press Game

Press a floating 3D button to increase a score counter.

1. Project Content

This 3D Button Press Game is built using the Ursina game engine. Players click a floating 3D sphere to increment their score. The game features dynamic button spawning, animated UI, sound effects, and win/lose states with a Play Again option.

2. Project Code

```
from ursina import *

from ursina.lights import DirectionalLight, AmbientLight

from math import sin

import time, random


app = Ursina()


# ———————— Window Settings ————————

window.title            = "3D Button Press Extreme (Cube Button)"

window.fullscreen       = False

window.exit_button.visible = False

window.fps_counter.enabled = False


# ———————— Lighting & Sky ————————

AmbientLight(color=color.rgba(255,255,255, .6))

DirectionalLight(shadows=True, rotation=(30, -60, 45), color=color.rgb(255,255,240))

sky = Sky(texture='background.hdr')


# ———————— Ground Plane ————————

ground = Entity(
```

```
    model='plane',
    texture='grid.png',
    texture_scale=(10,10),
    scale=(50,1,50),
    rotation_x=90,
    y=-1
)

# ——————— Audio ———————
click_sfx = Audio('click.wav', autoplay=False, volume=.7)
win_sfx   = Audio('win.wav',   autoplay=False, volume=.8)
lose_sfx  = Audio('lose.wav',  autoplay=False, volume=.8)

# ——————— Game & UI ———————
score     = 0
WIN_SCORE  = 30
is_over    = False

score_text = Text(
    text=f'Score: {score}',
    position=(-.5, .45),
    scale=2.5,
    color=color.white,
    background=True
)
```

```
game_over_text = Text('Game Over', position=(0, .2), scale=4, color=color.red, enabled=False)

win_text      = Text('You Win!',   position=(0, .2), scale=4, color=color.gold, enabled=False)

play_again_btn = Button('Play Again', position=(0, -.1), scale=.2, color=color.azure, enabled=False)


def animate_score():

   score_text.animate_scale(score_text.scale * 1.3, duration=.1, curve=curve.out_expo)

   invoke(score_text.animate_scale, 2.5, duration=.2, delay=.1, curve=curve.in_expo)


# ———————— Callbacks ————————

def on_button_click():

   global score, is_over

   if is_over:

      return

   click_sfx.play()

   score += 1

   score_text.text = f'Score: {score}'

   animate_score()

   if score >= WIN_SCORE:

      win_game()

   else:

      spawn_button(delay=.1)


def input(key):

   if key == 'left mouse down' and not is_over:

      if not button.hovered:
```

```
            end_game()


def end_game():
    global is_over
    is_over = True
    lose_sfx.play()
    game_over_text.enabled = True
    play_again_btn.enabled = True


def win_game():
    global is_over
    is_over = True
    win_sfx.play()
    win_text.enabled       = True
    play_again_btn.enabled = True


def on_play_again():
    global score, is_over
    score = 0
    is_over = False
    score_text.text        = f'Score: {score}'
    game_over_text.enabled = False
    win_text.enabled       = False
    play_again_btn.enabled = False
    spawn_button()
```

```
play_again_btn.on_click = on_play_again

# ——————— Button Spawner ———————
def spawn_button(delay=0):
    invoke(_spawn, delay=delay)

def _spawn():
    global button
    if 'button' in globals():
        destroy(button)
    pos = Vec3(random.uniform(-3,3), 0, random.uniform(1,3))
    button = Entity(
        model='cube',            # <-- swapped sphere for cube
        texture='white_cube',    # give it a simple texture
        color=color.azure,
        scale=(0.7, 0.3, 0.7),   # a flatter “button” shape
        collider='box',
        position=pos,
        highlight_color=color.yellow
    )
    button.on_click = on_button_click

# ——————— Update ———————
def update():
    if not is_over and 'button' in globals():
        # bob up/down
```

```
    button.y        = sin(time.time()*2 + button.x) * 0.4
    # spin around Y
    button.rotation_y += time.dt * 90
    # slight tilt oscillation around X
    button.rotation_x = sin(time.time()*3) * 15

# ———————— Camera ————————
camera.position   = (0, 2, -8)
camera.rotation_x = 15

# ———————— Start ————————
spawn_button()
app.run()
```

3. Key Technologies

- Ursina Engine
- Python 3
- Panda3D via Ursina
- Game UI and Events
- Audio playback with Ursina.Audio

4. Description

The game spawns a 3D sphere at random positions within a defined range. Players click the sphere to increase their score. The sphere bobs and spins for visual interest. Upon reaching the win threshold or mis-clicking, the game shows appropriate end-state text and allows replay.

5. Output

Test Case 1: Click only spheres until reaching a score of 30 → Displays "You Win!" and a Play Again button.

Test Case 2: Mis-click once before reaching 30 → Displays "Game Over" and a Play Again button.

6. Further Research

- Add different button models and materials
- Implement dynamic difficulty (faster bob/spin over time)
- Introduce power-ups that modify scoring
- Save high scores to a local file or online leaderboard
- Port to mobile with touch input support